Finetuning / Transfer Learning + Multiclass vs Multilabel predictions in classification

Alexander Binder

August 18, 2025

know where to look for

- http://d2l.ai/chapter_convolutional-modern/index.html

Takeaway for this lesson:

- finetuning can be used for models with different types of inputs and multiple forward streams, e.g. image and text – but always bottom up: from one of the inputs until the first layer where weights cannot be loaded anymore (bcs one changed the network design at this point to either a completely different layer, or due to shape mismatch). after one such blocker-layer, it makes no sense to load weights further above
- finetuning has three flavours with respect to what gets trained after weights were loaded: train all layers, train only the top layer, train only the k top-most layers

The main lesson for deep learning

Do not train deep neural networks from scratch![a] Always initialize the NN with weights from similar tasks trained on a very large dataset,

[a] ML has always exceptions :) ... except your data is in the order of hundred thousands and more.

While for some tasks with precise knowledge training from scratch works, 99% fine tuning of all layers is better than training from scratch.

where to get and how ? `torchvision.models`
https://pytorch.org/docs/stable/torchvision/models.html

- MNIST and CIFAR-10 work without finetuning – they are misleading.
- Note the simplicity of the tasks: images with 28×28, or 32×32 have limited variability and complexity compared to larger images! MNIST and CIFAR-10 are very useful for testing small ideas, but they are outliers within practically relevant deep learning tasks.

Practice session: you will take a deep network (densenet or a mobilenet), initialize it with weights from a 1000 class imagenet task, and then retrain it for 102 flowers classes. Why one can re-use weights from 1000 object classes that are mostly things and animals for flowers? The low level filters likely will be very similar.

- What needs to be changed? The last layer: to the number of output classes in your problem instead of 1000.
- therefore: last layer will not use pretrained weights
- see Figure 14.2.1 in https://d2l.ai/d2l-en.pdf in Chapter 14

What parts here can profit from transfer learning?

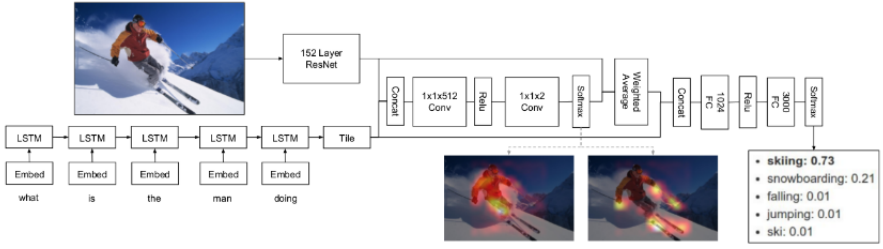
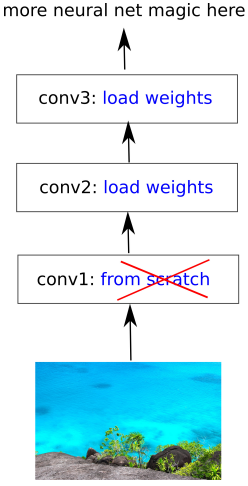


Figure 2. An overview of our model. We use a convolutional neural network based on ResNet [9] to embed the image. The input question is tokenized and embedded and fed to a multi-layer LSTM. The concatenated image features and the final state of LSTMs are then used to compute multiple attention distributions over image features. The concatenated image feature glimpses and the state of the LSTM is fed to two fully connected layers two produce probabilities over answer classes.

Kazemi and Elqursh https://arxiv.org/pdf/1704.03162.pdf

Above shows a VQA-architecture with attention. An image is processed by a CNN. A question is processed by embeddings, then an LSTM. The features are fused and weighted by attention layers. Final prediction is made by FC-layers with classification over possible answers as output.

It makes no sense to load weights for a layer, when one skips loading weights for any layer below. why ?



a neural net where finetuning makes NO sense –because we skip a layer early on.

Why does fine tuning help?

Alias	Network	# Parameters	Top-1 Accuracy	Top-5 Accuracy	Origin
alexnet	AlexNet	61,100,840	0.5492	0.7803	Converted from pytorch vision
densenet121	DenseNet-121	8,062,504	0.7497	0.9225	Converted from pytorch vision
densenet161	DenseNet-161	28,900,936	0.7770	0.9380	Converted from pytorch vision
densenet169	DenseNet-169	14,307,880	0.7617	0.9317	Converted from pytorch vision
densenet201	DenseNet-201	20,242,984	0.7732	0.9362	Converted from pytorch vision
inceptionv3	Inception V3 299x299	23,869,000	0.7755	0.9364	Converted from pytorch vision
mobilenet0.25	MobileNet 0.25	475,544	0.5185	0.7608	Trained with script
mobilenet0.5	MobileNet 0.5	1,342,536	0.6307	0.8475	Trained with script
mobilenet0.75	MobileNet 0.75	2,601,976	0.6738	0.8782	Trained with script
mobilenet1.0	MobileNet 1.0	4,253,864	0.7105	0.9006	Trained with script
mobilenetv2_1.0	MobileNetV2 1.0	3,539,136	0.7192	0.9056	Trained with script
mobilenetv2_0.75	MobileNetV2 0.75	2,653,864	0.6961	0.8895	Trained with script
mobilenetv2_0.5	MobileNetV2 0.5	1,983,104	0.6449	0.8547	Trained with script

- Deep NNs: high dimensionality of their parameters. https://paddleclas.readthedocs.io/en/latest/models/ResNet_and_vd_en.html . Training 10+ million parameters with 1000 samples violates the golden rule. You will overfit for sure.

Why does fine tuning help?

Alias	Network	# Parameters	Top-1 Accuracy	Top-5 Accuracy	Origin
alexnet	AlexNet	61,100,840	0.5492	0.7803	Converted from pytorch vision
densenet121	DenseNet-121	8,062,504	0.7497	0.9225	Converted from pytorch vision
densenet161	DenseNet-161	28,900,936	0.7770	0.9380	Converted from pytorch vision
densenet169	DenseNet-169	14,307,880	0.7617	0.9317	Converted from pytorch vision
densenet201	DenseNet-201	20,242,984	0.7732	0.9362	Converted from pytorch vision
inceptionv3	Inception V3 299x299	23,869,000	0.7755	0.9364	Converted from pytorch vision
mobilenet0.25	MobileNet 0.25	475,544	0.5185	0.7608	Trained with script
mobilenet0.5	MobileNet 0.5	1,342,536	0.6307	0.8475	Trained with script
mobilenet0.75	MobileNet 0.75	2,601,976	0.6738	0.8782	Trained with script
mobilenet1.0	MobileNet 1.0	4,253,864	0.7105	0.9006	Trained with script
mobilenetv2_1.0	MobileNetV2 1.0	3,539,136	0.7192	0.9056	Trained with script
mobilenetv2_0.75	MobileNetV2 0.75	2,653,864	0.6961	0.8895	Trained with script
mobilenetv2_0.5	MobileNetV2 0.5	1,983,104	0.6449	0.8547	Trained with script

- You can learn filters well only when you have enough training samples, often one needs $100k++$.
- Non-convex optimization problem: optimum depends on initialization. When having only a few thousand samples it is best to start from a good initialization – loading weights does that.

But why it is a good initialization?

But why it is a good initialization?

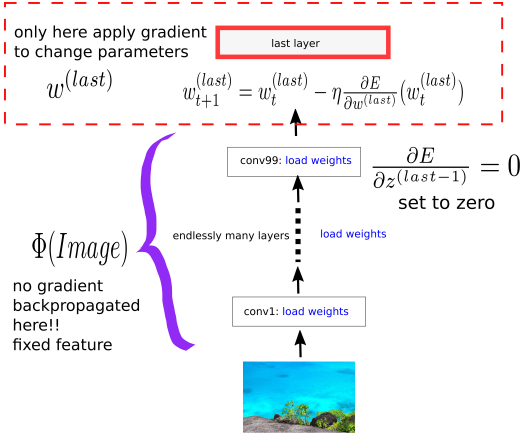
- empirical evidence: low-level features in deep networks learnt over wide and general tasks (e.g. Imagenet) can be reused for many other tasks, even with strange color distributions or geometrical tasks

Why does fine tuning help?

- A good initialization from finetuning will be destroyed when trained too long with too little samples.
- In practice backpropagating gradients changes the highest level weights faster (due to vanishing gradients), so that – at the beginning of training – the weights in the upper layers adapt faster towards what one wants to learn – and the overfitting by changing lower layer weights to bad optima sets in only later.

A good initialization from finetuning will be destroyed when trained too long... ?

- Finetuning has a special case: when the number of training data is very small, then one may want to retrain only the top layers.
- Finetuning in the narrowest sense: only train the top-layer. Works best when the number of training samples is very small.



Here an example when you retrain only the last layer as an extreme case of finetuning. This is often shown in tutorials

- training only top layers can be better for very small data sizes
- training all layers can be better for larger data sizes ... check on validation data
- if training only top-layers: without data augmentation can precompute bottom features for a speed up
  (but usually data augmentation improves test error! ... tradeoff speed vs performance)

Takeaway points

at the end of this lecture you should be able to:

- When using deep neural nets, always start from pretrained weights.

Outline

If you would not finetune, how to do it right then ?

- http://neuralnetworksanddeeplearning.com/chap3.html#weight_initialization
- https://arxiv.org/pdf/1502.01852.pdf

$$\mathrm{ReLU}(x) = \max(0, x)$$

need to initialize parameters w of a neural network. These are current guidelines:

Initialization for ReLU networks

- set biases to zero $b = 0$
- initialize weights as random values for symmetry breaking
- conv layers: draw weights from a normal with standard deviation equal to $\sigma = \sqrt{2}\frac{1}{\sqrt{n}}$, n the number of inputs
  $w_d \sim N(0, \sigma^2)$

- older Xavier-Glorot style initialization is considered obsolete for deep networks while it still works for shallow networks

$$\mathrm{PReLU}(x) = \max(0, x) + w * \min(0, x), w \text{ is trainable}$$

Initialization for pReLU networks

- set biases to zero $b = 0$
- initialize weights as random values for symmetry breaking
- conv layers: draw weights from a normal with standard deviation equal to $\sigma = \sqrt{\frac{2}{1+w^2}} \frac{1}{\sqrt{n}}$ where w is the negative slope, n the number of inputs
  $w_d \sim N(0, \sigma^2)$

If neurons in one layer and their input neurons receive the same weight, then their forward pass values might be the same and gradient updates might be the same.

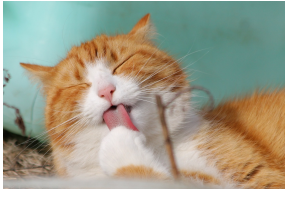
Then these neurons learns to encode the same thing.

Neural Net initialization: in PyTorch?

https://pytorch.org/docs/stable/nn.init.html

Two types of classification problems:

Multi-class




exactly one of cat or mouse

Multi-label




none / cat / mouse / cat+mouse

Multi-class vs Multi-label classification

- A classification problem is multi-class, if for every sample exactly one ground truth class is present. The ground truth can be represented by a one-hot label.
- A classification problem is multi-label, if for every sample zero to C ground truth classes can be present. The ground truth can be represented by either the zero vector or a sum of one-hot labels.

- Most benchmark datasets are multiclass
- Most real-life classification problems are multi-label[1]
- the difference affects how to compute probabilities for the output, how to measure error, how to define training loss

[1] if one just needs to account for absence of anything, one can add a background class in multi-class setups

- multiclass: i-th one-hot vector. exactly one entry is one, all others are zero.

$$e^{(i)} = (0, \ldots, 0, \underbrace{1}_{i}, 0, \ldots, 0)$$

$$(1,0,0,0,0),\ (0,1,0,0,0),\ (0,0,1,0,0),\ (0,0,0,1,0),\ (0,0,0,0,1)$$

- multilabel: valid multilabel ground truths: (zero or sum of one-hot)

$$(0,0,0,0,0)$$
$$(0,1,0,0,0)$$
$$(0,0,1,0,1)$$
$$(1,1,1,1,1)$$

Next steps:

- real-valued outputs for multi-class and multi-label
- probability outputs for multi-class and multi-label
- decision making for multi-class and multi-label probabilities
- loss functions for multi-class and multi-label

Suppose we have C classes. Want to produce one output for each class.

- for an affine prediction over inputs $\mathbf{x} \in \mathbb{R}^{(d,1)}$ ($\mathbf{x}$.shape$= (d, 1)$):

$$\mathbf{f}(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b},\ \mathbf{W}.shape = (c, d), \mathbf{b}.shape = (c, 1)$$

- sometimes, we want some non-linear model for prediction. Let us assume that that we did some non-linear feature extraction $\mathbf{h}(\mathbf{x})$
  (e.g. $\mathbf{h}(\mathbf{x}) =$ a pretrained resnet over an image $\mathbf{x}$ until the pooling layer)
  resulting in features with dimensionality $= e$, that is $\mathbf{h}(\mathbf{x}).shape = (e, 1)$
- for an affine prediction over some extracted features $\mathbf{h}(\mathbf{x})$ over $\mathbf{x}$:

$$\mathbf{f}(\mathbf{x}) = \mathbf{W}\mathbf{h}(\mathbf{x}) + \mathbf{b},\ \mathbf{W}.shape = (c, e), \mathbf{b}.shape = (c, 1)$$

- same for multiclass and multilabel

Recap: matrix multiplication $\mathbf{AB}$ is defined if $\mathbf{A}.shape = (c, e)$ and $\mathbf{B}.shape = (e, k)$

- multiclass: use softmax for the vector of all outputs
- multilabel: use one sigmoid for each output, but **no softmax**

Softmax for 2d inputs

The function

$$s(z_0, z_1) = \left(\frac{e^{z_0}}{e^{z_0} + e^{z_1}}, \frac{e^{z_1}}{e^{z_0} + e^{z_1}} \right)$$

is the Softmax for 2-dim inputs.
It maps any 2-dim-vector (z_0, z_1) onto a vector of positive numbers which sum up to 1.

$$\text{note: } s_0(z_0, z_1) = \frac{e^{z_0}}{e^{z_0} + e^{z_1}} \to P(Y = 0 | Z = (z_0, z_1))$$

$$s_1(z_0, z_1) = \frac{e^{z_1}}{e^{z_0} + e^{z_1}} \to P(Y = 1 | Z = (z_0, z_1))$$

Softmax for general inputs

The function

$$s(z_0, z_1, z_2, \ldots, z_{d-1}) = \left(\frac{e^{z_0}}{e^{z_0} + e^{z_1} + \ldots + e^{z_{d-1}}}, \frac{e^{z_1}}{e^{z_0} + e^{z_1} + \ldots + e^{z_{d-1}}}, \ldots, \right)$$
$$= \left(\frac{e^{z_0}}{\sum_{k=0}^{d-1} e^{z_k}}, \frac{e^{z_1}}{\sum_{k=0}^{d-1} e^{z_k}}, \frac{e^{z_2}}{\sum_{k=0}^{d-1} e^{z_k}}, \ldots, \frac{e^{z_{d-1}}}{\sum_{k=0}^{d-1} e^{z_k}} \right)$$

is the Softmax for d-dimensional inputs.
It maps any d-dim vector $(z_0, z_1, z_2, \ldots, z_{d-1})$ onto a vector $s(z_0, z_1, z_2, \ldots, z_{d-1})$ of positive numbers which sum up to 1.

Note: its i-th output component is

$$s_i(z_0, z_1, z_2, \ldots, z_{d-1}) = \frac{e^{z_i}}{\sum_{k=0}^{d-1} e^{z_k}} \text{ a model for } P(Y = i | Z = (z_0, z_1, z_2, \ldots, z_{d-1}))$$

Use case for softmax:

- multi-class problem, mutually exclusive labels
- needs to map any vector $(v_0, \ldots, v_{K-1})$ of length K on something which can be interpreted as discrete probability $P(k)$ over K elements (non-neg, sums up to 1 over all K classes)

Next: Why we are not using softmax for multi-label problems?

When is this a suitable prediction model?

Consider the setup of two output functions $f_0(x), f_1(x)$, with application of the softmax on top

When is this a good prediction model? $Y = 0$ - fish, $Y = 1$ gold

- do we expect the presence of gold versus fish mutually exclusively ?
- do we expect the presence of both gold and fish ?
- do we expect the absence of both gold and fish ?

When is this a suitable prediction model?

Consider the setup of two output functions $f_0(x), f_1(x)$, with application of the softmax on top

When is this a good prediction model? $Y = 0$ - fish, $Y = 1$ gold

- do we expect the presence of gold versus fish mutually exclusively ?
- do we expect the presence of both gold and fish ?
- do we expect the absence of both gold and fish ?

$$\text{Note one thing: } s(z_0, z_1) = \left(\frac{e^{z_0}}{e^{z_0} + e^{z_1}}, \frac{e^{z_1}}{e^{z_0} + e^{z_1}} \right)$$

If $z_0 \nearrow \infty$, then $P(Y = 0|X = x) \nearrow 1$ and $P(Y = 1|X = x) = \frac{e^{z_1}}{e^{z_0} + e^{z_1}} \searrow 0$

- increasing the value for one input decreases the probability of the class of the other input

Consider the setup of two output functions $f_0(x), f_1(x)$, with application of the softmax on top

- can we predict the presence of gold versus fish mutually exclusively ? YES
  $P(Y = 1|X = x) + P(Y = 0|X = x) = 1$
- can we predict the presence of both gold and fish ? NO
  $P(Y = 1|X = x) + P(Y = 0|X = x) = 1$ - both $Y = 0$, $Y = 1$ cannot have high probability close to 1 at the same time.
- can we predict the absence of both gold and fish ? NO
  $P(Y = 1|X = x) + P(Y = 0|X = x) = 1$ - both cannot have low probability close to 0 at the same time.

Above model makes sense to model the presence of two classes where one of them has to be present, while the other has to be absent

The multi-label case:

What is if we have images that can show gold and fishes together or images without both of them ?

How to model this ?

The multi-label case:

What is if we have images that can show gold and fishes together or images without both of them ?

How to model this ? Use one sigmoid for each output:

$$\begin{aligned}
\mathbf{f}(\mathbf{x}) &= \mathbf{W}\mathbf{h}(\mathbf{x}) + \mathbf{b}, \;\; \mathbf{W}.shape = (c, e), \mathbf{b}.shape = (c, 1) \\
z_i = f_i(\mathbf{x}) &= \mathbf{W}[\mathbf{i}, :]\mathbf{h}(\mathbf{x}) + b_i \\
p_i(\mathbf{x}) &= \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \\
&= \sigma(f_i(\mathbf{x})) = \frac{1}{1 + e^{-f_i(\mathbf{x})}}
\end{aligned}$$

Properties: $p_i(\mathbf{x}) \in (0, 1)$ but

$\sum_{c=0}^{C-1} p_c(\mathbf{x}) \neq 1$ – no competition between probabilities of different classes !

The multi-label case: How to model this ? We assume that we extracted some features $\mathbf{h}(\mathbf{x})$

Use one sigmoid for each output:

$$\begin{aligned}
\mathbf{f}(\mathbf{x}) &= \mathbf{W}\mathbf{h}(\mathbf{x}) + \mathbf{b}, \ \mathbf{W}.shape = (c, e), \mathbf{b}.shape = (c, 1) \\
z_i = f_i(\mathbf{x}) &= \mathbf{W}[\mathbf{i}, :]\mathbf{h}(\mathbf{x}) + b_i \\
p_i(\mathbf{x}) &= \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \\
&= \sigma(f_i(\mathbf{x})) = \frac{1}{1 + e^{-f_i(\mathbf{x})}}
\end{aligned}$$

Properties: $p_i(\mathbf{x}) \in (0, 1)$ but

$$\sum_{c=0}^{C-1} p_c(\mathbf{x}) \neq 1$$ – no competition between probabilities of different classes !

- now all classes can be close to 0 or close to 1 at the same time !! (if $\mathbf{h}(\mathbf{x}) \neq \mathbf{0}$)

The multi-label case: We assume that we extracted some features $\mathbf{h}(\mathbf{x})$. Then use one sigmoid for each output:

$$\begin{aligned}
\mathbf{f}(\mathbf{x}) &= \mathbf{W}\mathbf{h}(\mathbf{x}) + \mathbf{b}, \;\; \mathbf{W}.shape = (c, e), \mathbf{b}.shape = (c, 1) \\
z_i = f_i(\mathbf{x}) &= \mathbf{W}[\mathbf{i}, :]\mathbf{h}(\mathbf{x}) + b_i \\
p_i(\mathbf{x}) &= \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \\
&= \sigma(f_i(\mathbf{x})) = \frac{1}{1 + e^{-f_i(\mathbf{x})}}
\end{aligned}$$

Properties: $p_i(\mathbf{x}) \in (0, 1)$ but

$$\sum_{c=0}^{C-1} p_c(\mathbf{x}) \neq 1$$ – no competition between probabilities of different classes !

- now all classes can be close to 0 or close to 1 at the same time !!

$$\text{set } W[i, :] := -c\mathbf{h}(\mathbf{x}), \text{ then } \lim_{c \to +\infty} p_i(\mathbf{x}) = 0$$

$$\text{set } W[i, :] := +c\mathbf{h}(\mathbf{x}), \text{ then } \lim_{c \to +\infty} p_i(\mathbf{x}) = 1$$

set $W[i,:] := +c\mathbf{h}(\mathbf{x})$ then $\lim_{c\to+\infty} p_i(\mathbf{x}) = 1$

$$p_i(\mathbf{x}) = \sigma(f_i(\mathbf{x})) = \frac{1}{1+e^{-f_i(\mathbf{x})}} = \frac{1}{1+e^{-c\mathbf{h}(\mathbf{x})\cdot\mathbf{h}(\mathbf{x})-b}} = \frac{1}{1+e^{-c\|h(x)\|^2-b}}$$

$$\lim_{c\to+\infty} \frac{1}{1+e^{-c\|h(x)\|^2-b}} = \frac{1}{1+e^{-\infty}} = \frac{1}{1+0}$$

- multiclass: use softmax for the vector of all outputs

$$\mathbf{f}(\mathbf{x}) = \mathbf{W}\mathbf{h}(\mathbf{x}) + \mathbf{b}, \ \mathbf{W}.shape = (c, e), \mathbf{b}.shape = (c, 1)$$
$$p_i(\mathbf{x}) = s_i(\mathbf{f}(\mathbf{x})) \text{ i-th component of softmax function} \rightarrow P(Y = i|X = \mathbf{x})$$

  - probabilities compete among each other, sum up over all classes to 1

- multilabel: use one sigmoid for each output, but **no softmax**

$$\mathbf{f}(\mathbf{x}) = \mathbf{W}\mathbf{h}(\mathbf{x}) + \mathbf{b}, \ \mathbf{W}.shape = (c, e), \mathbf{b}.shape = (c, 1)$$
$$p_i(\mathbf{x}) = \sigma(f_i(\mathbf{x})) = \frac{1}{1 + e^{-f_i(\mathbf{x})}} \rightarrow P(Y = i|X = \mathbf{x})$$

  - probabilities are independent, can be all 1 or 0 at the same time

!!!

Advice: do not make hard decisions,
better order samples according to predicted probabilities !!!

... but if you have to ...

- multiclass:
  choose the one with the largest predicted probability

$$\text{class}(\mathbf{x}) = \text{argmax}_{i \in C} p_i(\mathbf{x})$$

- multilabel:
  make an independent decision for every class (whether something is present or not):

$$\text{class}_i(\mathbf{x}) = 1 \text{ if } p_i(\mathbf{x}) > 0.5$$

What do you want?

Goals

multi-class case:

- low loss if the model output score $f_c(x)$ for the (only) ground-truth class $(c : y_i^{(c)} = 1)$ is high
- high loss if the model output score $f_c(x)$ for the (only) ground-truth class is low

Extend the cross entropy-loss idea to C classes by following the neg-log approach:

- let be x_i, y_i a labeled sample. $y_i = (0, 0, 0, 1, 0, 0, 0, 1)$ is a one-hot-vector with c-th component being $y_i^{(c)}$

$$L(f(x_i), y_i) = \sum_c -y_i^{(c)} \ln s_c(x_i)$$

- note: only one of the $y_i^{(c)}$ will be 1. This is again: neg-log probability of the ground truth class

Now sum over all samples (indexed by i) in the training set

$$L(f(x_i), y_i) = \sum_c -y_i^{(c)} \ln s_c(x_i)$$

Multiclass crossentropy loss

Multiclass crossentropy loss is usable for multi-class problems. If the labels are one-hot labels, then it is the neg log probability of the ground truth class.

What do you want?

- you have to consider every output as a binary classification problem, and aggregate the losses over all c outputs (... sum it)

Goals

multi-label case:

- low loss if the model output score $f_c(x)$ for $c : y_i^{(c)} = 1$ is high
- low loss if the model output score $f_c(x)$ for $c : y_i^{(c)} = 0$ is low
- high loss if the model output score $f_c(x)$ for $c : y_i^{(c)} = 1$ is low
- high loss if the model output score $f_c(x)$ for $c : y_i^{(c)} = 0$ is high
- aggregate losses over all classes c

- You have C independent sigmoid outputs $s_c(x)$, for every of those C you use a binary cross-entropy loss , and sum over all classes
- let be x_i, y_i a labeled sample. $y_i = (1, 0, 0, 1, 0, 0, 0, 1)$ is a zero-1 vector with c-th component being $y_i^{(c)}$
- y_i is **not a one-hot vector**, all $y_i^{(c)}$ can be zero, or multiple 1s can be present
- the loss is: binary cross-entropy losses , summed over all classes

$$L(f(x_i), y_i) = \sum_c -y_i^{(c)} \ln s_c(x_i) - (1 - y_i^{(c)}) \ln(1 - s_c(x_i))$$

- note: $y_i^{(c)}$ can be 0 or 1, for every c. So it is c independent problems, for which you need c losses which cover the case of $y_i^{(c)} = 0$ and $y_i^{(c)} = 1$. Thats why both components in the sum.

Now sum over all samples (indexed by i) in the training set

$$L(f(x_i), y_i) = \sum_c -y_i^{(c)} \ln s_c(x_i) - (1 - y_i^{(c)}) \ln(1 - s_c(x_i))$$

class-summed binary crossentropy loss

Class-summed binary crossentropy loss is usable for multi-label problems.
If the labels are 0 or 1 for every class c, then

$$-y_i^{(c)} \ln s_c(x_i) - (1 - y_i^{(c)}) \ln(1 - s_c(x_i))$$

is the neg-log probability for the binary classification problem associated with class c.
($y_i^{(c)} = 0$ means that the class c is not present in sample x_i,
$y_i^{(c)} = 1$ means that the class c is present in sample x_i)

You can use other losses than cross-entropy. The idea remains the same:

- for multilabel problems you have to sum c independent loss terms for binary classifications.
- for multiclass problems you need a loss term for a multi-class classification loss for c classes:
  - low loss if the model output score for the ground-truth class ($c : y_i^{(c)} = 1$) is high
  - high loss if the model output score for the ground-truth class is low

You can use other losses than cross-entropy. Example for another loss for multi-label predictions:

- suppose your outputs are real numbers, not probabilities

$$f_c(x) = w^{(c)} \cdot x + b^{(c)} \in (-\infty, +\infty)$$

- now use for each class the hinge loss

$$\max(0, 1 - (2y_i^{(c)} - 1) f_c(x_i))$$

- then the total loss (over all classes c and samples i) would be:

$$\frac{1}{N} \sum_{i=0}^{N-1} \frac{1}{C} \sum_{c=0}^{C-1} \max(0, 1 - (2y_i^{(c)} - 1) f_c(x_i))$$

You can use other losses than cross-entropy. Example for another loss for multi-label predictions:

- suppose your outputs are real numbers, not probabilities

$$f_c(x) = w^{(c)} \cdot x + b^{(c)} \in (-\infty, +\infty)$$

- now use for each class the hinge loss

$$\max(0, 1 - (2y_i^{(c)} - 1) f_c(x_i))$$

- when this would have zero loss ?
- case: $y_i^{(c)} = 1$
  - then $(2y_i^{(c)} - 1) = +1$, then the term is $\max(0, 1 - f_c(x_i))$
  - it has zero loss if $f_c(x_i) \geq 1$ (then $1-$ this is negative)

You can use other losses than cross-entropy. Example for another loss for multi-label predictions:

- suppose your outputs are real numbers, not probabilities

$$f_c(x) = w^{(c)} \cdot x + b^{(c)} \in (-\infty, +\infty)$$

- now use for each class the hinge loss

$$\max(0, 1 - (2y_i^{(c)} - 1) f_c(x_i))$$

- when this would have zero loss ?
- case: $y_i^{(c)} = 0$
  - then $(2y_i^{(c)} - 1) = -1$, then the term is $\max(0, 1 + f_c(x_i))$
  - it has zero loss if $f_c(x_i) \leq -1$ (then $1+$ this is negative)

Multi-class case:

- prediction output: Softmax, competitive
- training loss: multi-class type, e.g. multi-class cross-entropy

Multi-label case:

- prediction output: C sigmoids, independent
- training loss: C times binary type , aggregated, e.g. sum of C binary cross-entropy losses

We have seen this model in lecture 1:

$$f(x) = w \cdot x + b$$
$$\text{predicted label: } q(x) = 1[f(x) \geq 0]$$
$$s(x) = \frac{1}{1 + e^{(f(x))}}$$

How is this related to the case of the two models??

We have seen this model in lecture 1:

$$f(x) = w \cdot x + b$$
$$\text{predicted label: } q(x) = 1[f(x) \geq 0]$$
$$s(x) = \frac{1}{1 + e^{(f(x))}} = P(Y = 1|X = x)$$

How is this related to the case of the two models??

$$f_0(x) = w^{(0)} \cdot x + b^{(0)}$$
$$f_1(x) = w^{(1)} \cdot x + b^{(1)}$$
$$\text{predicted label: } q(x) = 1[f_1(x) \geq f_0(x)]$$
$$s_i(x) = \frac{e^{(f_i(x))}}{e^{(f_0(x))} + e^{(f_1(x))}} = P(Y = i|X = x)$$

- how this is connected to the logistic regression with a single output?
- when it is a good idea to use this model?
- what loss can we use in this case with two outputs ?

$$f_0(x) = w^{(0)} \cdot x + b^{(0)}$$
$$f_1(x) = w^{(1)} \cdot x + b^{(1)}$$
$$\text{predicted label: } q(x) = 1[f_1(x) \geq f_0(x)] = P(Y = 1|X = x)$$

$$s_i(x) = \frac{e^{(f_i(x))}}{e^{(f_0(x))} + e^{(f_1(x))}} = P(Y = i|X = x)$$

- how this is connected to the logistic regression with a single output?

We had before: linear model with sigmoid

$$f(x) = w \cdot x + b$$
$$s(x) = \frac{1}{1 + e^{(-f(x))}} = P(Y = 1|X = x)$$

$$f_0(x) = w^{(0)} \cdot x + b^{(0)}$$
$$f_1(x) = w^{(1)} \cdot x + b^{(1)}$$
$$\text{predicted label: } q(x) = 1[f_1(x) \geq f_0(x)]$$
$$s_i(x) = \frac{e^{(f_i(x))}}{e^{(f_0(x))} + e^{(f_1(x))}} = P(Y = i|X = x)$$

- how this is connected to the logistic regression with a single output?
- take the above model and set $f(x) = f_1(x) - f_0(x) = (w^{(1)} - w^{(0)}) \cdot x + b^{(1)} - b^{(0)}$, plug this into the logistic sigmoid:

$$s(x) = \frac{1}{1 + e^{(-f(x))}} = \frac{1}{1 + e^{(-(f_1(x) - f_0(x)))}} = \frac{1}{1 + e^{(f_0(x) - f_1(x))}} = \frac{1}{1 + e^{(f_0(x) - f_1(x))}} \frac{e^{f_1(x)}}{e^{f_1(x)}}$$
$$= \frac{e^{(f_1(x))}}{e^{(f_1(x))} + e^{(f_0(x))}} = P(Y = 1|X = x) \text{ for the case of two affine models and softmax}$$

- set $f(x) = f_1(x) - f_0(x) = (w^{(1)} - w^{(0)}) \cdot x + b^{(1)} - b^{(0)}$, plug this into the logistic sigmoid $\sigma(z) = \frac{1}{1+e^{(z)}}$
- you obtain $\frac{e^{(f_1(x))}}{e^{(f_1(x))}+e^{(f_0(x))}} = P(Y = 1|X = x)$ - the softmax for the case of two models $f_0(x), f_1(x)$

Conclusion:
You can convert both setups one into the other , and they will predict still the same.

- take two models $f_0(x), f_1(x)$, plug their difference $f_1(x) - f_0(x)$ into the logistic sigmoid. Then this will still predict the same value as the softmax for the case of two models $f_0(x), f_1(x)$

- set $f(x) = f_1(x) - f_0(x) = (w^{(1)} - w^{(0)}) \cdot x + b^{(1)} - b^{(0)}$, plug this into the logistic sigmoid $\sigma(z) = \frac{1}{1+e^{(z)}}$
- you obtain $\frac{e^{(f_1(x))}}{e^{(f_1(x))}+e^{(f_0(x))}} = P(Y = 1|X = x)$ - the softmax for the case of two models $f_0(x), f_1(x)$

Conclusion:
You can convert both setups one into the other , and they will predict still the same.

- take the setup with the one model $f(x)$ and the sigmoid,
  - set $f_1(x) = f(x)$, $f_0(x) = 0$, apply the softmax, then this predicts the same value as the sigmoid would do:

$$\frac{e^{(f_1(x))}}{e^{(f_1(x))} + e^{(f_0(x))}} = \frac{e^{(f(x))}}{e^{(f(x))} + e^{(0)}} = \frac{e^{(f(x))}}{e^{(f(x))} + 1}$$

Now divide below and above by $e^{(f(x))}$:
$\frac{e^{(f_1(x))}}{e^{(f_1(x))}+e^{(f_0(x))}} = \frac{1}{1+e^{(-f(x))}}$. This is the sigmoid output

Conclusion:

Both setups can be converted into each other

Two setups:

- a single output function $f(x)$, apply the sigmoid on top
- two output functions $f_0(x), f_1(x)$, apply the softmax on top

You can convert both setups one into the other, and they will predict still the same.

How to arrive at drawing weights from a zero mean normal distribution with variance equal to $\frac{2}{n}$?

Motivation for the parameters

The parameters are chosen so that

- variances of feature maps in the forward pass are approximately equal across layers.
- variances of gradient norms in the backward pass are approximately equal across layers.

How to arrive at drawing weights from a zero mean normal distribution with variance equal to $\frac{2}{n}$?

Consider a linear neuron. $x^{(l)}$ are the inputs from a previous layer, which itself might be the outputs of a preceding relu on the layer $y^{(l-1)}$ which was computed before $y^{(l)}$:

$$y^{(l)} = \sum_{d=1}^{n^{(l)}} w_d^{(l)} x_d^{(l)} + b$$

$$x_d^{(l)} = \max(0, y^{(l-1)})$$

The basic idea is: initialize w_d such that the variance of outputs at initialization is preserved through layers:

$$Var(y^{(1)}) = Var(y^{(2)}) = \ldots = Var(y^{(l-1)}) = Var(y^{(l)})$$

Furthermore we assume:

- w_d will be drawn from a random distribution with zero mean $E[w_d] = 0$
- $b = 0$ at initialization
- x_d is statistically independent of w_d, thus $E[f(x_d)g(w_d)] = E[f(x_d)]E[g(w_d)]$

The old Xavier Glorot-style argument ignored the symmetry breaking by the relu, thus assumed

$$E[x^{(l)}] = E[y^{(l-1)}] = 0$$

(which makes sense for antisymmetric functions like the tanh). Furthermore it assumed $var(x) = var(y^{(l-1)})$. Under such an assumption

$$Var(wx) = Var(w)E[x^2] = Var(w)var(x) = Var(w)var(y^{(l-1)})$$

$$Var(y^{(l)}) = \prod_l \left(Var(w^l)n^{(l)}\right) Var(y^{(1)})$$

This results in a heuristic $Std(w^l) = \sqrt{\frac{1}{n^{(l)}}}$ which however does not work well when the neural network is stacked too deeply.